MSAI605 — Project Milestone 4 Profiling Report

Authors: Arun Kulkarni, Pramod Goyal **Course:** MSAI605

Milestone 4 — CPU Profiling Report

System under test: final embedding-based face verifier (FaceNet / InceptionResnetV1, VGGFace2 weights, 512-d L2-normalized embeddings, cosine similarity threshold 0.397).

Submission baseline: CPU-only.

Source artifacts: - profiling script: `scripts/profile_inference.py` - machine-readable summary: `outputs/profiling/cpu_profile_summary.json`

---

1. Measurement environment

Field	Value
OS	Windows 11 (10.0.26200)
CPU	AMD64 Family 25 Model 97 Stepping 2 (AuthenticAMD)
Logical cores	32
Python	3.13.1
PyTorch	2.11.0+cpu
NumPy	2.4.2
`torch.set_num_threads`	32 (set explicitly to use all logical cores)
Device	CPU only — no CUDA used at any point

The exact same hardware/software fingerprint is recorded inside `cpu_profile_summary.json` so the numbers in this report are traceable back to the run.

---

2. Methodology

- Timing primitive: `time.perf_counter()` (monotonic, highest resolution available).

- Each batch size is profiled with 3 warmup forward passes (discarded) followed by 10 timed repeats.
- For each repeat the script measures three stages plus end-to-end wall time:
- **Preprocess** — `src/embedder.py:preprocess_image` invoked once per image, then `torch.cat` to stack into a `(B, 3, 160, 160)` tensor. Performed for both sides of the pair.
- **Embed** — single forward pass `model(stack)` per side, under `torch.no_grad()`, followed by L2 normalization. Two forward passes per repeat (one for each side of the pair). This is where stacked batching takes effect.
- **Score** — vectorized cosine similarity (`src/similarity.py:cosine_similarity`), threshold comparison, and per-pair `compute_confidence` (`src/confidence.py`).
- Input source: `outputs/pairs_test.npz` (committed test split). 64 pairs are sampled per batch-size run.
- Final-system version profiled: FaceNet (InceptionResnetV1, VGGFace2), threshold 0.3969849246231156, embedding dim 512, `score_direction = "higher_is_same"`. These are pulled directly from `configs/inference_config.json` at runtime so the report cannot drift from the released configuration.
- The production path `src/inference.py:verify_pair` already exposes a per-stage `breakdown` that times exactly these three stages — the profiling script reuses the same component functions, with the only difference being a stacked forward pass instead of a single-image pass.

---

3. Per-stage latency (CPU baseline, batch size = 1)

This row is the **required CPU baseline submission result** for the final embedding-based system.

Stage	Mean (ms)	Std (ms)
Preprocess	1.162	0.468
Embed	79.957	4.572
Score	0.210	0.040
End-to-end	**81.331**	**4.729**

Per-pair throughput at batch size 1: **12.30 pairs/s**.

Embedding accounts for **~98%** of end-to-end latency. Preprocess and score together are under 1.5 ms.

---

4. CPU baseline (clearly labeled)

> **CPU baseline:** on the measurement environment described in §1, the final embedding-based face verifier processes a single pair in **81.3 ms end-to-end (mean over 10 repeats, std 4.7 ms)**, at a

> steady-state throughput of **12.30 pairs/s**. The decomposition is preprocess 1.16 ms / embed 79.96 ms / score 0.21 ms.

This is the baseline figure the System Card and README cite for operational-constraints discussion.

---

5. Batch-size sensitivity

Stacked batching: B images per side fused into a single forward pass. Same warmup/repeat policy across all sizes.

batch_size	preprocess_ms	embed_ms	score_ms	end_to_end_ms	throughput (pairs/s)
1	1.162 ± 0.468	79.957 ± 4.572	0.210 ± 0.040	81.331 ± 4.729	12.30
2	2.008 ± 0.358	84.607 ± 4.625	0.178 ± 0.025	86.795 ± 4.438	23.04
4	3.255 ± 0.198	94.649 ± 2.802	0.173 ± 0.015	98.078 ± 2.799	40.78
8	6.191 ± 0.423	123.926 ± 6.700	0.178 ± 0.010	130.297 ± 6.644	61.40
16	12.543 ± 1.198	183.999 ± 5.803	0.204 ± 0.031	196.747 ± 6.487	81.32

Interpretation

- **Throughput scales sub-linearly but monotonically** from 12.3 → 81.3 pairs/s as B goes from 1 to 16 — a **6.6× speedup at 16× the batch size**. The marginal benefit shrinks at larger batches because per-image embedding cost is already small relative to fixed forward-pass overhead.
- **Per-pair embedding cost drops sharply** with batch size: ~80 ms/pair at B=1, ~42 ms/pair at B=2, ~24 ms/pair at B=4, ~15 ms/pair at B=8, ~11.5 ms/pair at B=16. This is the expected amortization of model forward-pass overhead over more images.
- **Preprocess scales linearly with batch** (1.2 → 12.5 ms across 1 → 16). It is implemented serially in Python (PIL resize per image), so each image adds a roughly constant cost. This is intentional — preprocess is not a bottleneck at any tested size.
- **Score stage is essentially flat** at ~0.18–0.21 ms regardless of batch size, consistent with a vectorized NumPy einsum over a tiny `(B, 512)` array.
- **No memory issues** were encountered up to B=16 on this machine. Batch sizes beyond 16 were not tested for the submission baseline; given the trend, additional speedup past B=16 is expected to be small relative to the latency growth per call.

6. Interpretation and implications

- **Embedding dominates.** Across every tested batch size, the FaceNet forward pass accounts for >93% of end-to-end latency. Preprocess and score are negligible by comparison. Any meaningful CPU-side speedup of the released system would have to target the embedding step — for example via ONNX export, INT8 quantization, or a smaller backbone (MobileFaceNet) — not the pre/post stages.
- **Single-pair latency is the binding constraint for interactive use.** ~81 ms per pair on this CPU is acceptable for an offline verifier or a low-throughput interactive CLI but is the ceiling on real-time per-call latency without architectural changes.
- **Batching is the right lever for throughput.** When a workload can submit pairs in groups, batch size 8–16 yields 5–7× throughput at the cost of ~2.4× higher per-call wall time. For this release the production CLI (`scripts/verify.py`) intentionally remains single-pair to keep the per-call interface simple; the batched path is only used for profiling, not for production semantics.
- **The released system is operationally CPU-bound on convolutions.** This is consistent with InceptionResnetV1 having ~24M parameters and the architecture being convolution-heavy. The profile contains no surprises — no anomalous variance, no Windows-specific stalls in the timed region.

7. Reproducing this report

```
# uv (recommended)
uv run python scripts/profile_inference.py \
   --device cpu \
   --output outputs/profiling/cpu_profile_summary.json

# venv + pip
python scripts/profile_inference.py \
   --device cpu \
   --output outputs/profiling/cpu_profile_summary.json
```

The script regenerates `outputs/profiling/cpu_profile_summary.json` and a `cpu_profile_summary.md` sidecar table. All numbers in this report are pulled verbatim from those artifacts.